



Jerry Lin <jerry@museder.com>

[WordPress Plugin Directory] Review in Progress: Museder RestoreOne

WordPress.org Plugin Directory <plugins@wordpress.org>

2026年2月7日 晚上8:43

收件者: Jerry Lin <jerry@museder.com>

🙌 Thank you for the changes "artherslin"!

Your plugin is not yet ready to be approved, you are receiving this email because the volunteers have manually checked it and have found some issues in the code / functionality of your plugin.

Please check this email thoroughly, address any issues listed, test your changes, and upload a corrected version of your code if all is well.

List of issues found

The URL(s) declared in your plugin seems to be invalid or does not work.

From your plugin:

Plugin URI: <https://musederlabs.com/> - museder-restoreone.php - Connection timed out after 5001 milliseconds

Undocumented use of a 3rd Party / external service

Plugins are permitted to require the use of third party/external services as long as they are clearly documented.

When your plugin reach out to external services, you must disclose it. This is true even if you are the one providing that service.

You are required to document it in a clear and plain language, so users are aware of: **what data is sent, why, where and under which conditions.**

To do this, you must update your readme file to **clearly explain that your plugin relies on third party/external services**, and include **at least the following information** for each third party/external service that this plugin uses:

- What the service is and what it is used for.
- What data is sent and when.
- Provide links to the service's terms of service and privacy policy.

Remember, this is for your own legal protection. Use of services must be upfront and well documented. This allows users to ensure that any legal issues with data transmissions are covered.

Example:

== External services ==

This plugin connects to an API to obtain weather information, it's needed to show the weather information and forecasts in the included widget.

It sends the user's location every time the widget is loaded (If the location isn't available and/or the user hasn't given their consent, it displays a configurable default location).

This service is provided by "PRT Weather INC": [terms of use](#), [privacy policy](#).

Example(s) from your plugin:

```
# Domain(s) mentioned in the readme file. Links to service terms and/or privacy policy not found.
includes/class-cloud-service.php:220 : sprintf( 'https://s3.%s.amazonaws.com', $settings['s3_region'] );
includes/class-backup-lite-s3-service.php:851 $host = 's3.' . $region . '.amazonaws.com';
includes/class-backup-lite-s3-service.php:849 $host = $bucket . 's3.' . $region . '.amazonaws.com';
```

Calling core loading files directly

Calling core files like wp-config.php, wp-blog-header.php, wp-load.php directly via an include is not permitted.

These calls are prone to failure as not all WordPress installs have the exact same file structure. In addition it opens your plugin to security issues, as WordPress can be easily tricked into running code in an unauthenticated manner.

Your code should always exist in functions and be called by action hooks. This is true even if you need code to exist outside of WordPress. Code should only be accessible to people who are logged in and authorized, if it needs that kind of access. Your plugin's pages should be called via the dashboard like all the other settings panels, and in that way, they'll always have access to WordPress functions.

<https://developer.wordpress.org/plugins/hooks/>

There are some exceptions to the rule in certain situations and for certain core files. In that case, we expect you to use `require_once` to load them and to use a function from that file immediately after loading it.

If you are trying to "expose" an endpoint to be accessed directly by an external service, you have some options.

- You can expose a 'page' use [query_vars](#) and/or [rewrite rules](#) to create a virtual page which calls a function. [A practical example](#).
- You can create an [AJAX endpoint](#).
- You can create a [REST API endpoint](#).

Example(s) from your plugin:

```
includes/class-restore-handler.php:509 require_once ABSPATH . 'wp-admin/includes/media.php';
```

Determine files and directories locations correctly

WordPress provides several functions for easily determining where a given file or directory lives.

We detected that **the way your plugin references some files, directories and/or URLs may not work with all WordPress setups**. This happens because there are hardcoded references or you are using the WordPress internal constants.

Let's improve it, please check out the following documentation:

<https://developer.wordpress.org/plugins/plugin-basics/determining-plugin-and-content-directories/>

It contains all the functions available to determine locations correctly.

Most common cases in plugins can be solved using the following functions:

- For where your plugin is located: `plugin_dir_path()` , `plugin_dir_url()` , `plugins_url()`
- For the uploads directory: `wp_upload_dir()` (*Note: If you need to write files, please do so in a folder in the uploads directory, not in your plugin directories*).

Example(s) from your plugin:

```
includes/class-backup.php:4519 $prefixes[] = wp_normalize_path( trailingslashit( WP_LANG_DIR ) );
includes/class-settings.php:158 $user_file = trailingslashit( ABSPATH ) . 'wp-admin/includes/user.php';
includes/class-restore.php:300 $upgrade = rtrim( (string) ABSPATH, '\\') . '/wp-admin/includes/upgrade.
php';
templates/page-settings.php:24 $temp_dir = function_exists( 'get_temp_dir' ) ? get_temp_dir() : (
function_exists( 'sys_get_temp_dir' ) ? sys_get_temp_dir() : ABSPATH );
-----> ABSPATH
```

Don't Force Set PHP Limits Globally

While many plugins can need optimal settings for PHP, we ask you please not set them as global defaults.

Having defines like `ini_set('memory_limit', '-1');` run globally (like on init or in the `__construct()` part of your code) means you'll be running that for everything on the site, which may cause your users to fall out of compliance with any limits or restrictions on their host.

If you must use those, you need to limit them specifically to only the exact functions that require them.

Example(s) from your plugin:

```
includes/class-ai1wm-converter.php:572 set_time_limit(600);
includes/class-restore-handler.php:2464 set_time_limit(0);
includes/class-ai1wm-converter.php:633 set_time_limit(600);
includes/class-restore-service.php:2714 set_time_limit(600);
includes/class-restore-handler.php:708 set_time_limit(600);
includes/class-restore-handler.php:300 set_time_limit(600);
includes/class-restore-service.php:291 set_time_limit(600);
includes/class-backup.php:4037 set_time_limit(0);
```

... out of a total of 10 incidences.

Nonces and User Permissions Needed for Security

Please add a nonce check to your input calls (`$_POST`, `$_GET`, `$REQUEST`) to prevent unauthorized access.

If you use `wp_ajax_` to trigger submission checks, remember they also need a nonce check.



Checking permissions: Keep in mind, a nonce check alone is not bulletproof security. Do not rely on nonces for authorization purposes. When needed, use it together with `current_user_can()` in order to prevent users without the right permissions from accessing things they shouldn't.

Also make sure that the nonce logic is correct by making sure it cannot be bypassed. Checking the nonce with `current_user_can()` is great, but mixing it with other checks can make the condition more complex and, without realising it, bypassable, remember that **anything** can be sent through an input, **don't trust any input**.

Keep performance in mind. Don't check for post submission outside of functions. Doing so means that the check will run on every single load of the plugin, which means that every single person who views any page on a site using your plugin will be checking for a submission. This will make your code slow and unwieldy for users on any high traffic site, leading to instability and eventually crashes.

The following links may assist you in development:

- <https://developer.wordpress.org/plugins/security/nonces/>
- <https://developer.wordpress.org/plugins/javascript/ajax/#nonce>
- <https://developer.wordpress.org/plugins/settings/settings-api/>

From your plugin:

```

includes/pro/reports-controller.php:203 Museder_Restoreone_Reports_Controller::handle_report_download()
[classMethod] No nonce check found validating input origin on lines 203–218
# ↳ Line 210: $file = sanitize_text_field( wp_unslash( $_GET['file'] ?? '' ) );
includes/class-restore-handler.php:1574 Museder_Restoreone_Restore_Handler::chunk_finalize() [classMethod]
No nonce check found validating input origin on lines 1574–1587
# ↳ Line 1579: $session_id = isset( $_POST['session_id'] ) ? sanitize_text_field( wp_unslash(
$_POST['session_id'] ) ) : '';
includes/class-log-handler.php:246 Museder_Restoreone_Log_Handler::ajax_download_log() [classMethod] No
nonce check found validating input origin on lines 246–255
# ↳ Line 252: $log = isset( $_GET['log'] ) ? sanitize_text_field( wp_unslash( $_GET['log'] ) ) : '';
# ↳ Line 255: $log = sanitize_text_field( wp_unslash( $_GET['amp;log'] ) );
includes/class-restore-handler.php:1363 Museder_Restoreone_Restore_Handler::chunk_prepare() [classMethod]
No nonce check found validating input origin on lines 1363–1409
# ↳ Line 1370: $filename = sanitize_file_name( wp_unslash( $_POST['filename'] ) );
# ↳ Line 1376: $filesize = absint( wp_unslash( $_POST['filesize'] ) );
# ↳ Line 1382: $chunk_size = absint( wp_unslash( $_POST['chunk_size'] ) );
# ↳ Line 1388: $total_chunks = absint( wp_unslash( $_POST['total_chunks'] ) );
includes/class-ui.php:831 Museder_Restoreone_UI::handle_delete_backup() [classMethod] No nonce check
found validating input origin on lines 831–841
# ↳ Line 835: $filename = isset( $_POST['filename'] ) ? sanitize_text_field( wp_unslash( $_POST['filename'] ) ) :
'';

```

... out of a total of 9 incidences.

Please, make sure that the nonce logic is correct. It's important to be cautious when structuring conditional checks around nonces.

```

includes/ai/class-ai-rest-controller.php:79 if ( '' === $nonce || ! wp_verify_nonce( $nonce, 'wp_rest' ) ) {

```

Processing the whole input

We strongly recommend you **never** attempt to process the whole `$_POST/$_REQUEST/$_GET` stack. This makes your plugin slower as you're needlessly cycling through data you don't need. Instead, you should only be attempting to process the items within that are required for your plugin to function.

Example(s) from your plugin:

```

includes/class-settings.php:248 $data = file_get_contents( 'php://input' );
includes/class-schedule-handler.php:966 $raw = file_get_contents( 'php://input' ); // Fallback for JSON
payload.

```

Generic function/class/define/namespace/option names

All plugins must have unique function names, namespaces, defines, class and option names. This prevents your plugin from conflicting with other plugins or themes. We need you to update your plugin to use more unique and distinct names.

A good way to do this is with a prefix. For example, if your plugin is called "Museder RestoreOne" then you could use names like these:

- `function musere_save_post(){ ... }`
- `class MUSERE_Admin { ... }`
- `update_option( 'musere_options', $options );`
- `add_shortcode( 'musere_shortcode', $callback );`
- `register_setting( 'musere_settings', 'musere_user_id', ... );`
- `define( 'MUSERE_PLUGIN_DIR', plugin_dir_path( __FILE__ ) );`
- `global $musere_options;`
- `add_action('wp_ajax_musere_save_data', ... );`

- namespace artherslin\musederrestoreone;

Disclaimer: These are just examples that may have been self-generated from your plugin name, we trust you can find better options. If you have a good alternative, please use it instead, this is just an example.

The prefix should be **at least four (4) characters long** (don't try to use two- or three-letter prefixes anymore). We host almost 100,000 plugins on WordPress.org alone. There are tens of thousands more outside our servers. Believe us, you're likely to encounter conflicts.

You also need to avoid the use of `__` (double underscores), `wp_`, or `_` (single underscore) as a prefix. Those are reserved for WordPress itself. You can use them inside your classes, but not as stand-alone function.

Please remember, if you're using `_n()` or `__()` for translation, that's fine. We're **only** talking about functions you've created for your plugin, not the core functions from WordPress. In fact, those core features are why you need to not use those prefixes in your own plugin! You don't want to break WordPress for your users.

Related to this, using `if (!function_exists('NAME')) {` around all your functions and classes sounds like a great idea until you realize the fatal flaw. If something else has a function with the same name and their code loads first, your plugin will break. Using `if-exists` should be reserved for shared libraries only.

Remember: Good prefix names are unique and distinct to your plugin. This will help you and the next person in debugging, as well as prevent conflicts.

Analysis result:

```
# This plugin is using the prefixes "museder", "musederrestoreone", "musederrestoreonev2",
"musederrestoreonepro", "musederrestoreoneadmin", "musederrestoreonerestore",
"musederrestoreonereports", "musederrestoreoneadminui" for 169 element(s).
```

```
# Looks like there are elements not using common prefixes.
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

Note: Options and Transients must be prefixed.

This is really important because the options are stored in a shared location and under the name you have set. If two plugins use the same name for options, they will find an interesting conflict when trying to read information introduced by the other plugin.

Also, once your plugin has active users, changing the name of an option is going to be really tricky, so let's make it robust from the very beginning.

Example(s) from your plugin:

```
includes/class-backup-jobs.php:979 add_option($key, $value, '', 'no');
```

```
includes/class-backup-jobs.php:962 add_option($key, $value, '', 'no');
```

 **Continue with the review process.**

Read this email thoroughly.

Please, take the time to fully understand the issues we've raised. Review the examples provided, read the relevant documentation, and research as needed. Our goal is for you to gain a clear understanding of the problems so you can address them effectively and avoid similar issues when maintaining your plugin in the future.

Note that there may be false positives – we are humans and make mistakes, we apologize if there is anything

we have gotten wrong. If you have doubts you can ask us for clarification, when asking us please be clear, concise, direct and include an example.

Complete your checklist.

- ✓ I fixed all the issues in my plugin based on the feedback I received and my own review, as I know that the Plugins Team may not share all cases of the same issue. I am familiar with tools such as Plugin Check, PHPCS + WPCS, and similar utilities to help me identify problems in my code.
- ✓ I tested my updated plugin on a clean WordPress installation with [WP_DEBUG set to true](#).

 **Do not skip this step.** Testing is essential to make sure your fixes actually work and that you haven't introduced new issues.

- ✓ I acknowledge that this review will be rejected if I overlook the issues or fail to test my code.
- ✓ I went to ["Add your plugin"](#) and uploaded the updated version. *I can continue updating the code there throughout the review process — the team will always check the latest version.*
- ✓ I replied to this email. I was concise and shared any clarifications or important context that the team needed to know.
I didn't list all the changes, as the team will review the entire plugin again and that is not necessary at all.

 To make this process as quick as possible and to avoid burden on the volunteers devoting their time to review this plugin's code, we ask you to thoroughly check all shared issues and fix them before sending the code back to us. *I know we already asked you to do so, and it is because we are really trying to make it very clear.*

While we try to make our reviews as exhaustive as possible we, like you, are humans and may have missed things. We appreciate your patience and understanding.

Review ID: R museder-restoreone/artherslin/1Dec25/T8 7Feb26/3.8RC3

--

WordPress Plugins Team | plugins@wordpress.org

<https://make.wordpress.org/plugins/>

<https://developer.wordpress.org/plugins/wordpress-org/detailed-plugin-guidelines/>

<https://wordpress.org/plugins/plugin-check/>

On Tue, Feb 3, 2026 at 5:15 PM UTC, WordPress.org Plugin Directory <plugins@wordpress.org> wrote:

This is an automated message to confirm that we have received your updated plugin file.

File updated by artherslin, version 2.7.241.

Comment: Subject: Re: [WordPress Plugin Directory] Review in Progress: Museder RestoreOne (Review ID: R museder-restoreone/artherslin/1Dec25/T7 23Jan26/3.8RC2)

Hello WordPress Plugins Team,

Thank you for the detailed feedback. I have addressed the issues and uploaded an updated version. Here is my reply:

--- Issues addressed ---

1. AI generated output

I have removed the file you mentioned (docs/wp-compliance-checklist.md). In the latest package, the entire docs/ folder has been excluded from the plugin zip so that no development/AI-generated documentation is shipped to users.

2. Generic function/class/define/namespace/option names

- All options, transients, and cron hook names now use the prefix "museder_restoreone_" (or the same in constants/classes where applicable).
- The previous "backup_lite" identifiers are only referenced in the main plugin file for one-time migration (renaming old options/crons to the new prefixed names and clearing old cron hooks). No new options or hooks use